

Unit 1: Lesson 1

Introduction to Data Structures



definition :- data structure is a way of organizing and storing data in a computer program so that it can be accessed and used efficiently. Data structures provide a means of managing large amounts of data, enabling efficient searching, sorting, insertion and deletion of data.

Data :- Fact that can be recorded or stored

->e.g. Person Name, Age, Gender and Weight etc.

Structure :- The way something is arranged or organized.

Data Structure = Data (Values) + Structure (Organization)

Type of data structure

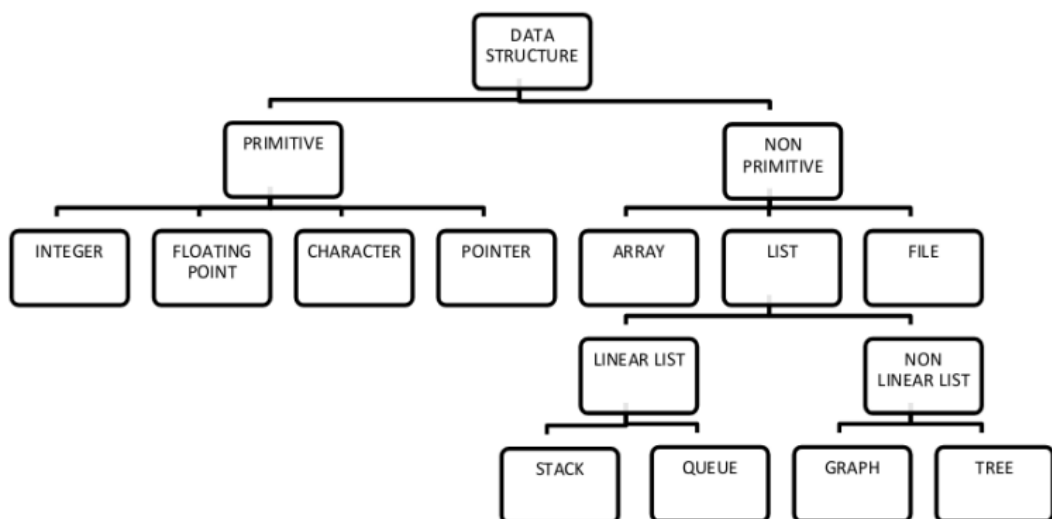
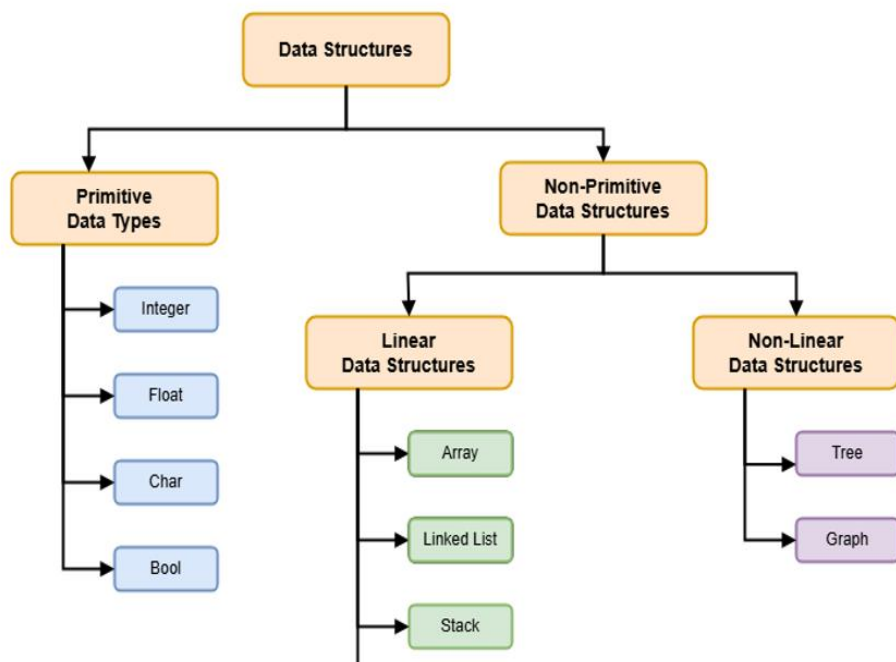
Primitive data structures are the most basic data structures available in a programming language, such as integers, floating-point numbers, characters and Booleans.

-> the most basic data structures available in a programming language, such as integers, floating-point numbers, characters and Booleans.

Non-primitive data structures are complex data structures that are built using non - primitive data types, such as arrays, linked lists, stacks, queues, trees, graphs and hash tables.

Following terms are the foundation terms of a data structure.

- **Interface** – Each data structure has an interface. Interface represents the set of operations that a data structure supports. An interface only provides the list of supported operations, type of parameters they can accept and return type of these operations.
- **Implementation** – Implementation provides the internal representation of a data structure. Implementation also provides the definition of the algorithms used in the operations of the data structure.



Characteristics of a Data Structure

- **Correctness** – Data structure implementation should implement its interface correctly.
- **Time Complexity** – Running time or the execution time of operations of data structure must be as small as possible.
- **Space Complexity** – Memory usage of a data structure operation should be as little as possible.

Importance of Data Structures in Programming

- **Efficient data access and manipulation:** Data structures enable quick access and manipulation of data. For example, an array allows constant-time access to elements using their index, while a hash table allows fast access to elements based on their key. Without data structures, programs would have to search through data sequentially, leading to slow performance.
- **Memory management:** Data structures allow efficient use of memory by allocating and deallocating memory dynamically. For example, a linked list can dynamically allocate memory for each element as needed, rather than allocating a fixed amount of memory upfront. This helps avoid memory wastage and enables efficient memory management.

- **Code reusability:** Data structures can be reused across different programs and projects. For example, a generic stack data structure can be used in multiple programs that require LIFO (Last-In-First-Out) functionality, without having to rewrite the same code each time.
- **Optimization of algorithms:** Data structures help optimize algorithms by enabling efficient data access and manipulation. For example, a binary search tree allows fast searching and insertion of elements, making it ideal for implementing searching and sorting algorithms.
- **Scalability:** Data structures enable programs to handle large amounts of data effectively. For example, a hash table can store large amounts of data while providing fast access to elements based on their key.

Data structures can broadly be classified into two main categories:

1. Primitive Data Structures Primitive data structures are the most basic types of data structures provided by programming languages.

They include:

- Integer: Whole numbers (e.g., 1, 2, 100).
- Float: Numbers with decimal points (e.g., 3.14, 0.001).
- Character: Single characters (e.g., 'A', 'B').
- Boolean: Logical values (true or false).

Sample Program in C++ (Primitive Data Types)

```
#include using namespace std;
int main() {
int num = 10; // Integer
float pi = 3.14; // Float
char letter = 'A'; // Character
bool isHappy = true; // Boolean
cout << "Integer: " << num << endl;
cout << "Float: " << pi << endl;
cout << "Character: " << letter << endl; cout << "Boolean: " << isHappy << endl; return 0;
}
```

2. Non-Primitive Data Structures

Non-primitive data structures are more complex and are built using primitive data types. They are further divided into:

a. Linear Data Structures

In linear data structures, elements are arranged sequentially, and each element has a unique predecessor and successor (except the first and last elements).

- Array: A collection of elements of the same type, stored in contiguous memory locations.
- Linked List: A collection of nodes where each node contains data and a pointer to the next node.
- Stack: A collection of elements following the Last In First Out (LIFO) principle.
- Queue: A collection of elements following the First In First Out (FIFO) principle.

Sample Program in C++ (Array)

```
#include<iostream>
using namespace std;
int main() {
int arr[5] = {1, 2, 3, 4, 5}; // Array of integers
cout << "Array elements:" << endl;
for (int i = 0; i < 5; i++) {
cout << arr[i] << " ";
}
cout << endl;
return 0; }
```

b. Non-Linear Data Structures

In non-linear data structures, elements are not arranged sequentially but follow a hierarchical or interconnected structure.

- **Tree:** A hierarchical structure where each node has a parent node and zero or more child nodes.
- **Graph:** A set of nodes (vertices) connected by edges, representing relationships.

Sample Program in C++ (Basic Tree Traversal)

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = right = nullptr;
    }
};

void inOrder(Node* root) {
    if (root == nullptr) return;
    inOrder(root->left);
    cout << root->data << " ";
    inOrder(root->right);
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);

    cout << "In-order Traversal: ";
    inOrder(root);
    cout << endl;

    return 0;
}
```

Operations on different Data Structure:

- **Traversing:** Traversing a Data Structure means to visit the element stored in it. It visits data in a systematic manner. This can be done with any type of DS.
- **Searching:** Searching means to find a particular element in the given data-structure. It is considered as successful when the required element is found. Searching is the operation which we can performed on data-structures like array, linkedlist, tree, graph, etc.
- **Insertion:** It is the operation which we apply on all the data-structures. Insertion means to add an element in the given data structure. The operation of insertion is successful when the required element is added to the required data-structure. It is unsuccessful in some cases when the size of the data structure is full and when there is no space in the data-structure to add any additional element. The insertion has the same name as an insertion in the data-structure as an array, linked-list, graph, tree. In stack, this operation is called Push. In the queue, this operation is called Enqueue.
- **Deletion:** It is the operation which we apply on all the data-structures. Deletion means to delete an element in the given data structure. The operation of deletion is successful when the required element is deleted from the data structure. The deletion has the same name as a deletion in the data- structure as an array, linked-list, graph, tree, etc. In stack, this operation is called Pop. In Queue this operation is called Dequeue.
- **Create:** It reserves memory for program elements by declaring them. The creation of data structure can be done during
 - o Compile-time
 - o Run-time.

- **Selection:** It selects specific data from present data. You can select any specific data by giving condition in loop.
- **Update:** It updates the data in the data structure. You can also update any specific data by giving some condition in loop like select approach.
- **Sort:** Sorting data in a particular order (ascending or descending). We can take the help of many sorting algorithms to sort data in less time. Example: bubble sort which takes $O(n^2)$ time to sort data. There are many algorithms present like merge sort, insertion sort, selection sort, quick sort, etc.
- **Merge:** Merging data of two different orders in a specific order may ascend or descend. We use merge sort to merge sort data.
- **Split Data:** Dividing data into different sub-parts to make the process complete in less time.

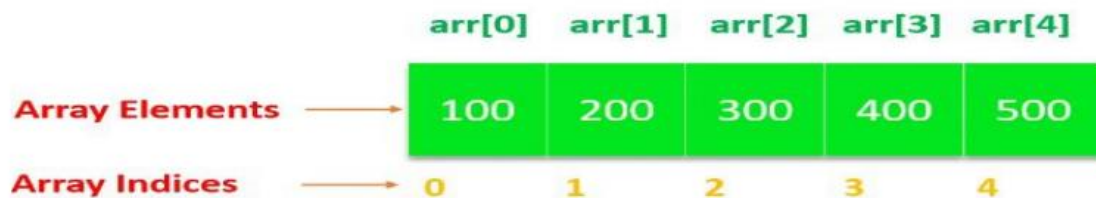
ARRAYS:

Definition:- An array is a linear data structure that collects elements of the same data type and stores them in contiguous and adjacent memory locations

Types of arrays

Array in C are of two types; Single dimensional arrays and Multidimensional arrays.

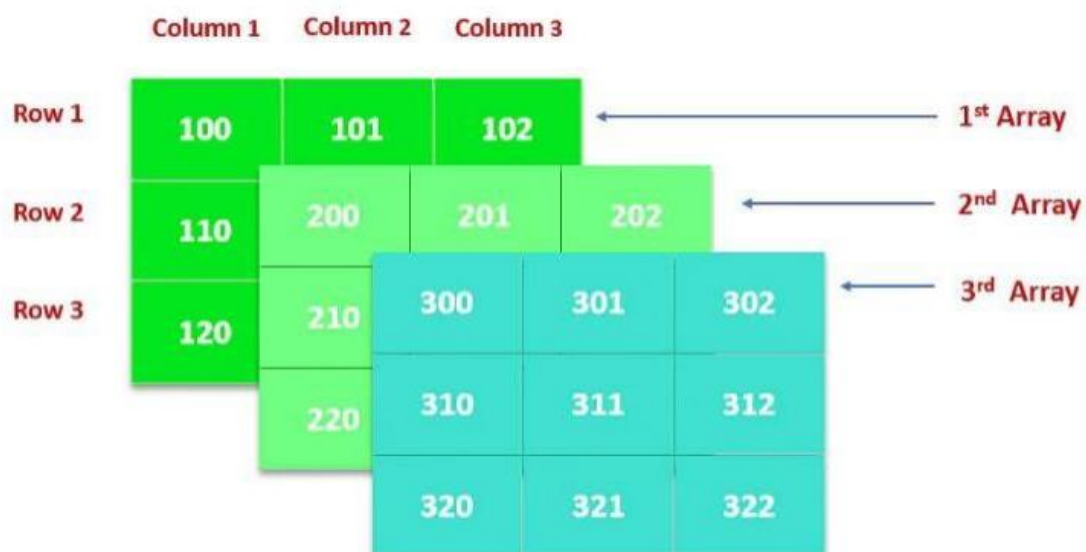
- **Single Dimensional Arrays:** Single dimensional array or 1-D array is the simplest form of arrays that can be found in C. This type of array consists of elements of similar types and these elements can be accessed through their.



ARRAY IN C

Multi-dimensional Arrays:

The most common type of multi-dimensional array that is used in the C language is a 2-D array. However, the number of dimensions can be more than 2 depending upon the compiler of the user's system. These arrays consist of elements that are array themselves.



OPERATION ON ARRAYS

There are a number of operations that can be performed on an array which are:

1. Traversal
2. Sorting
3. Insertion
4. Deletion
5. Searching
6. Merging

Traversal: Traversal means accessing each array element for a specific purpose, either to perform an operation on them.

Sorting: It is a process of arranging the elements in the array.

Insertion: it is a process of adding a new element to the array in the specified location.

Deletion: it is a process of removing an element from the array from the specified location.

Searching: it is a process of finding an element in an array.

Merging: it is a process of combining the elements of two array.

Declaration of array

1. `int a[ ];`

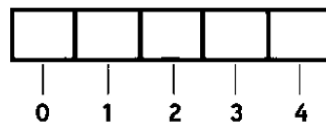
Error

(can't be empty).

2. `int a[5];`

- Size must be a natural number.
- Total number of variables in array
- Not an index

3. `int a[5];`



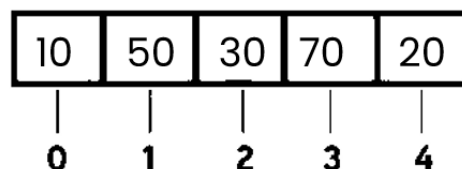
Whatever is the size of an array it always consumes memory in a sequential fashion.

Local array when not initialized, it contains garbage values.

Initialization of Array

1. You can initialize array during declaration

```
int a[5] = { 10, 50, 30, 70, 20 };
```



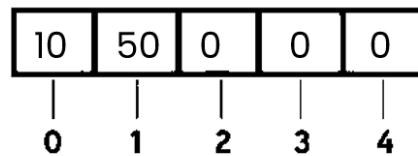
2. You cannot initialize an array during declaration more than its size.

```
int a[5] = { 10, 50, 30, 70, 20, 80, 40 };
```

Error

3. You can initialize an array during declaration less than its size.

```
int a[5] = { 10, 50 };
```



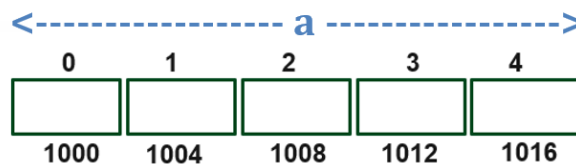
Remaining variables in array will contain 0 and not garbage value.

4. During declaration you can leave [] empty only when you initialize array at the same time

```
int a[ ] = { 10, 50, 30, 80, 20 };
```

```
int a[5];
```

$a \approx 1000$



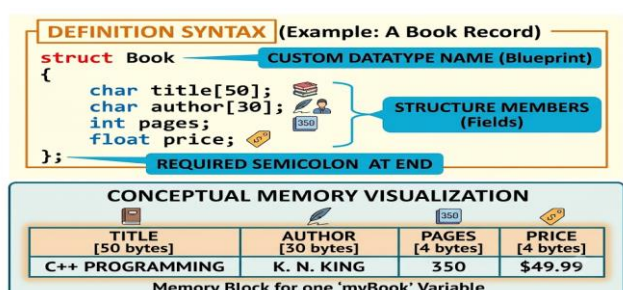
[a\[0\]](#)
[a\[1\]](#)
[a\[2\]](#)
[a\[3\]](#)
[a\[4\]](#)

Note:

Array access is **fast** because accessing any element by index takes **constant time $O(1)$** .

Structures

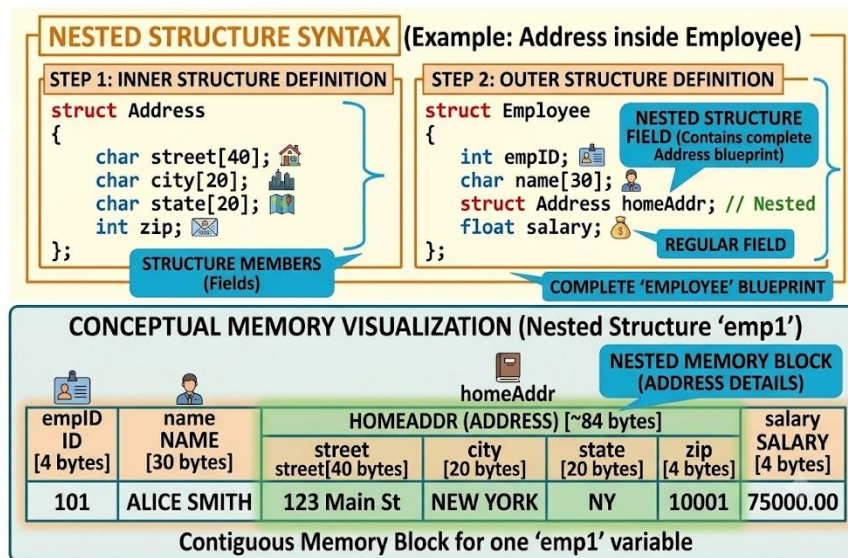
A **structure** is a user-defined data type that groups variables of different data types under a single name. Structures are ideal for representing real-world entities, such as a book with an ID, title, and price



Why Use Structures?

- To represent real-world entities like Student, Employee, Book, etc.
- To group related information together.
- Useful when multiple variables are logically connected.

NESTED STRUCTURE



A **Pointer to a Structure** is simply a pointer variable that holds the **memory address** of a struct variable, rather than holding a primitive type like int or char.

🔗 Why Do We Use It? (The "Why")

1. **Efficiency (Pass by Reference):** Passing a large structure into a function copies the *entire block of memory*. Passing a pointer copies only the **memory address** (typically 4 or 8 bytes), saving memory and processing time.
2. **Dynamic Memory Allocation:** Used with functions like malloc() or new to create structure variables on the heap dynamically.
3. **Building Data Structures:** Essential for creating linked structures like **Linked Lists**, **Trees**, and **Graphs**, where each node contains a pointer to another node.

🔑 Key Syntax & Operators

1. Declaration & Initialization

```
struct Student s1 = {101, "Vishal"}; // Regular structure variable
struct Student *ptr = &s1;           // Pointer storing the address of s1
```

2. Accessing Members (The Arrow Operator ->)

When you have a regular structure variable, you use the **dot operator (.)**.

When you have a pointer to a structure, you use the **arrow operator (->)**.

ptr->member ≡ (*ptr).member

```

#include <stdio.h>

struct Student {
    int id;
    float marks;
};

int main() {
    struct Student s1 = {1, 95.5};    // Normal structure variable
    struct Student *ptr = &s1;        // Pointer to structure

    // Accessing via dot operator
    printf("ID via Dot: %d\n", s1.id);

    // Accessing via Arrow Operator (Preferred)
    printf("ID via Pointer: %d\n", ptr->id);
    printf("Marks via Pointer: %.1f\n", ptr->marks);

    // Alternative dereferencing syntax: (*ptr).id
    // Note: Parentheses around *ptr are required because '.' has higher precedence
    printf("ID via Dereferencing: %d\n", (*ptr).id);

    return 0;
}

```

UNION

a union is a user-defined datatype in which we can define members of different types of data types just like structures. But one thing that makes it different from structures is that the member variables in a union share the same memory location, unlike a structure that allocates memory separately for each member variable. The size of the union is equal to the size of the largest data type.

☐ Memory space can be used by one member variable at one point in time, which means if we assign value to one member variable, it will automatically deallocate the other member variable stored in the memory which will lead to loss of data.

☐ Need of Union in C++

- o When the available memory is limited, it can be used to achieve memory efficiency.
- o It is used to encapsulate different types of data members.
- o It helps in optimizing the performance of applications.

Syntax of Union in C++

- **Union** is defined using the 'union' keyword.

```

union Union_Name {
    // Declaration of data members
};
union_variables;

```

After defining the union we can also create an instance of it the same as we create an object of the class or declare any other data type.

C++

```

int main()
{
    union Union_Name var1;
    // or

```

```
Union_Name var1;  
}
```

- **Example of Union in C++**

```
#include <iostream>  
using namespace std;  
union geek          // Creating a union  
{  
    // Defining data members  
    int age;  
    char grade;  
    float GPA;  
};  
int main()  
{  
    // Defining a union variable  
    union geek student1;  
    // Assigning values to data member of union geek and  
    // printing the values of data members  
    student1.age = 25;  
    cout << "Age : " << student1.age << endl;  
    student1.grade = 'B';  
    cout << "Grade : " << student1.grade << endl;  
    student1.GPA = 4.5;  
    cout << "GPA : " << student1.GPA << endl;  
    return 0;  
}
```

Pointer in C++

- Pointers are symbolic representations of addresses. They enable programs to simulate call-by-reference as well as to create and manipulate dynamic data structures. Iterating over elements in arrays or other data structures is one of the main use of pointers.
- The address of the variable you're working with is assigned to the pointer variable that points to the same data type (such as an int or string).
- **Syntax:**

C++

datatype *var_name;

int *ptr; // ptr can point to an address which holds int data

- How to use a pointer?
 - Define a pointer variable
 - Assigning the address of a variable to a pointer using the unary operator (&) which returns the address of that variable.
 - Accessing the value stored in the address using unary operator (*) which returns the value of the variable located at the address specified by its operand.
- The reason we associate data type with a pointer is that it knows how many bytes the data is stored in. When we increment a pointer, we increase the pointer by the size of the data type to which it points. To master the use of pointers and their applications, explore the C++ Course for comprehensive lessons and hands-on examples.

- **Example**

```
#include <iostream>

using namespace std;

int main()
{
    int var = 20;

    // declare pointer variable
    int* ptr;

    // note that data type of ptr and var must be same
    ptr = &var;

    // assign the address of a variable to a pointer
    cout << "Value at ptr = " << ptr << "\n";
    cout << "Value at var = " << var << "\n";
    cout << "Value at *ptr = " << *ptr << "\n";
    return 0;
}
```

- **Array Name as Pointers**

- An array name contains the address of the first element of the array which acts like a constant pointer. It means, the address stored in the array name can't be changed. For example, if we have an array named val then val and &val[0] can be used interchangeably.

// C++ program to illustrate Array Name as Pointers

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    // Declare an array
```

```
    int val[3] = { 5, 10, 20 };
```

```
    // declare pointer variable
```

```
    int* ptr;
```

```
    // Assign the address of val[0] to ptr
```

```
    // We can use ptr=&val[0];(both are same)
```

```
    ptr = val;
```

```
    cout << "Elements of the array are: ";
```

```
    cout << ptr[0] << " " << ptr[1] << " " << ptr[2];
```

```
    return 0;
```

```
}
```

Advantages of Pointers

- Pointers reduce the code and improve performance. They are used to retrieve strings, trees, arrays, structures, and functions.
- Pointers allow us to return multiple values from functions.
- In addition to this, pointers allow us to access a memory location in the computer's memory.

Key Concepts in Memory Representation:

- Top: Pointer or index indicating the current top element of the stack.
- Stack Overflow: Occurs when attempting to push an element onto a full stack (array implementation).
- Stack Underflow: Occurs when attempting to pop an element from an empty stack.

Basic Operations

Stack operations may involve initializing the stack, using it and then de-initializing it. Apart from these basic stuffs, a stack is used for the following two primary operations –

- push() – Pushing (storing) an element on the stack.
- pop() – Removing (accessing) an element from the stack.

When data is PUSHed onto stack.

To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks –

- peek() – get the top data element of the stack, without removing it.
- isFull() – check if stack is full.
- isEmpty() – check if stack is empty.

C++ Classes

A class in C++ is a user-defined data type that combines data members and member functions into a single unit. It acts as a blueprint for creating objects and helps organize code in an object-oriented way.

- **Data Members:** Variables inside the class used to store data.
- **Member Functions:** Functions inside the class used to perform operations on the data.

```
#include <iostream>
using namespace std;

class Car {
public:
    string brand;

    void display() {
        cout << "Car Brand: " << brand;
    }
};

int main() {
    Car car1;

    car1.brand = "Toyota";
    car1.display();

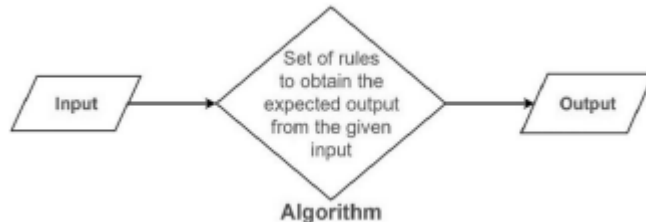
    return 0;
}
```

Algorithm

Specifications:

What is an Algorithm? Algorithm Basics

The word Algorithm means "A set of finite rules or instructions to be followed in calculations or other problem-solving operations" or "A procedure for solving a mathematical problem in a finite number of steps that frequently involves recursive operations". Therefore, Algorithm refers to a sequence of finite steps to solve a particular problem.



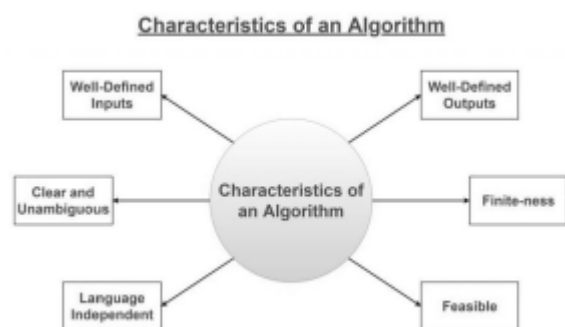
It can be understood by taking the example of cooking a new recipe. To cook a new recipe, one reads the instructions and steps and executes them one by one, in the given sequence. The result thus obtained is the new dish is cooked perfectly. Every time you use your phone, computer, laptop or calculator you are using Algorithms. Similarly, algorithms help to do a task programming to get the expected output.

The Algorithm designed is language-independent, i.e. they are just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.

What is the need for algorithms?

Algorithms are necessary for solving complex problems efficiently and effectively.

1. They help to automate processes and make them more reliable, faster and easier to perform.
2. Algorithms also enable computers to perform tasks that would be difficult or impossible for humans to do manually.
3. They are used in various field such as mathematics, computer science, engineering, finance and many others to optimize cases, analyse data, make predictions and provide solutions to problems.



As one would not follow any written instructions to cook the recipe, but only the standard one. Similarly, not all written instructions for programming is an algorithm. In order for some instructions to be an algorithm, it must have the following characteristics:

- **Clear and Unambiguous:** The algorithm should be clear and unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- **Well-Defined Inputs:** If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
- **Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
- **Feasible:** The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.
- **Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.
- **Input:** An algorithm has zero or more inputs. Each that contains a fundamental operator must accept zero or more inputs.
- **Output:** An algorithm produces at least one output. Every instruction that contains a fundamental operator must accept

zero or more inputs.

- **Definiteness:** All instructions in an algorithm must be unambiguous, precise, and easy to interpret. By referring to any of the instructions in an algorithm can clearly understand what is to be done. Every fundamental operator in instruction must be defined without any ambiguity.
- **Finiteness:** An algorithm must terminate after a finite number of steps in all test cases. Every instruction which contains a fundamental operator must be terminated within a finite amount of time. Infinite loops or recursive functions without base conditions do not possess finiteness.
- **Effectiveness:** An algorithm must be developed by using very basic, simple, and feasible operations so that one can trace it out by using just paper and pencil.

Properties of Algorithm:

- It should terminate after a finite time.
- It should produce at least one output.
- It should take zero or more input.
- It should be deterministic means giving the same output for the same input case.
- Every step in the algorithm must be effective i.e. every step should do some work

Advantages of Algorithms:

- It is easy to understand.
- An algorithm is a step-wise representation of a solution to a given problem.
- In Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

Disadvantages of Algorithms:

- Writing an algorithm takes a long time so it is time-consuming.
- Understanding complex logic through algorithms can be very difficult.
- Branching and Looping statements are difficult to show in Algorithms.

Why Do We Need Algorithms?

Algorithms are essential for solving problems efficiently in computer science and beyond. Here's why:

- **Problem Solving:** Algorithms provide systematic ways to solve problems, like searching, sorting, or optimizing routes.
- **Efficiency:** Well-designed algorithms save time and resources (e.g., faster searches improve user experience).
- **Scalability:** Algorithms handle large datasets, critical for applications like Google Search or social media platforms.
- **Automation:** Algorithms automate repetitive tasks, reducing human effort (e.g., auto-correct in text editors).
- **Standardization:** Algorithms ensure consistent and repeatable results across different systems.

Introduction to Performance Analysis

What is Performance Analysis?

Performance analysis evaluates how efficiently an algorithm uses resources, primarily:

- **Time:** How long does the algorithm take to complete?
- **Space:** How much memory does the algorithm use?

Why Analyze Performance?

- **Choose the Best Algorithm:** Compare algorithms solving the same problem (e.g., linear search vs. binary search).
- **Optimize Applications:** Ensure software runs fast on large inputs.

- **Resource Constraints:** Mobile devices or embedded systems have limited memory and processing power.
- **Predict Scalability:** Understand how an algorithm performs as input size grows.

Time Complexity: Quantifies the **amount of time** an algorithm takes to run as a function of the input size (n). It measures operation counts, not actual execution time (which depends on CPU/hardware).

Space Complexity: Quantifies the total **amount of memory space** required by an algorithm to run to completion, including input space and auxiliary (extra) space.

Asymptotic Notations

- Big O ($\mathcal{O}$): Worst-case scenario (Upper bound). This is what we focus on most in computer science.
- Big Omega (Ω): Best-case scenario (Lower bound).
- Big Theta (Θ): Average-case scenario (Tight bound).

3. Common Time Complexities (Fastest to Slowest)

Notation	Name	Common Example
$\mathcal{O}(1)$	Constant Time	Accessing an array element by index
$\mathcal{O}(\log n)$	Logarithmic Time	Binary Search
$\mathcal{O}(n)$	Linear Time	Linear Search / Iterating an array once
$\mathcal{O}(n \log n)$	Linearithmic Time	Merge Sort / Quick Sort (average)
$\mathcal{O}(n^2)$	Quadratic Time	Bubble Sort / Nested Loops
$\mathcal{O}(2^n)$	Exponential Time	Recursive Fibonacci (naive)

